
USRT Online Phase

Joint Dynamic Programming for Time & Energy Slack Distribution

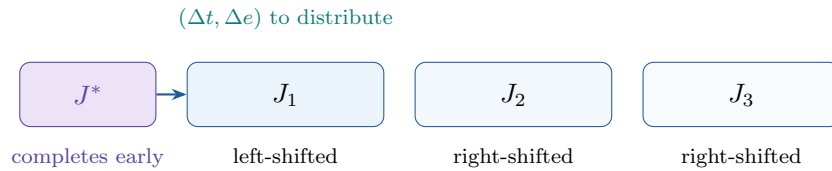
Design Document — Theory

The Online Problem:

Jobs finish earlier than worst-case ($WCET > ACET$), creating *time slack* Δt and *energy slack* Δe .

The Goal: Use this unexpected slack to execute additional optional segments and gain utility — deciding optimally, in microseconds, at every job completion.

The Method: A single 3-dimensional DP table $\mathcal{V}[j][\Delta t][\Delta e]$, precomputed offline, queried online.



Contents

1	The Online Problem	2
1.1	Where Slack Comes From	2
1.2	The Four Decision Options	2
2	The Joint DP Formulation	3
2.1	Why a Single Joint Table	3
2.2	The Geometry: Left-Shift J_1 , Right-Shift the Rest	3
2.3	The Recurrence	4
2.4	Mapping the Four Options to (a_t, a_e)	4
2.5	Non-Uniform Segment Sizes	4
3	Worked Example (3 Jobs, Non-Uniform Segments)	5
3.1	Job Parameters	5
3.2	Table $\mathcal{V}[3]$ — Last Job	5
3.3	Table $\mathcal{V}[2]$ — Job 2 (uses $\mathcal{V}[3]$)	5
3.4	Table $\mathcal{V}[1]$ — Job 1 (uses $\mathcal{V}[2]$)	6
3.5	Online Trace at $(\Delta t=4, \Delta e=5)$	6
4	Second Worked Example (4 Jobs, Verified)	7
4.1	Job Parameters	7
4.2	Table $\mathcal{V}[4]$ — Last Job (J_4)	7
4.3	Table $\mathcal{V}[3]$ — Job 3 (uses $\mathcal{V}[4]$)	7
4.4	Table $\mathcal{V}[2]$ — Job 2 (uses $\mathcal{V}[3]$)	8
4.5	Table $\mathcal{V}[1]$ — Job 1, the Entry Point (uses $\mathcal{V}[2]$)	8
4.6	Online Traces (All Verified)	8
4.7	What This Larger Example Demonstrates	10
5	The Global Energy vs Local Time Asymmetry	11
5.1	Statement of the Problem	11
5.2	Three Ways to Tackle the Asymmetry	11
5.2.1	Approach A — Per-Processor Time DP, Shared Energy Coupling	11
5.2.2	Approach B — Distribute the Energy Budget Offline (Per-Processor Energy)	11
5.2.3	Approach C — Utility-Density Arbitration for Shared Energy	12
5.3	Recommended Combination	12
6	Top 5 Concerns & Suggested Solutions	13
7	Efficiency: Making the DP Practical	15
7.1	Better Bounds	15
7.2	Aggregate States	15
7.3	Lazy DP	15
8	Summary	16

1. The Online Problem

1.1 Where Slack Comes From

The offline phase (ILP v2 or the heuristic) produces a complete schedule assuming **worst-case execution time (WCET)** for every job. Each job is assigned a processor $P_{i,j}^*$, a frequency f_z , and a segment count k .

At runtime, jobs almost always finish earlier than their WCET — they run for their **actual case execution time (ACET)**, which is typically well below WCET. This early completion produces two kinds of unexpected slack:

Slack Type	Time Slack Δt	Energy Slack Δe
Scope	Local — belongs to one processor	Global — one shared pool
Accumulation	Non-accumulative — bounded by deadlines, expires	Accumulative — carries forward indefinitely
Created when	A job finishes before its scheduled slot ends	A job consumes less energy than budgeted
Usable for	Running more optional segments now/soon; lowering frequency	Running more optional segments anywhere, anytime

This asymmetry is the central design tension. Energy slack is a global bank account — what one processor saves, any processor can spend, later or now. Time slack is a perishable, processor-local resource — if processor P_0 has 5 free units now, processor P_1 cannot borrow them, and P_0 itself loses them once the next deadline passes. Section 5 is devoted to handling this asymmetry.

1.2 The Four Decision Options

The paper (Section VI) defines exactly four ways to deploy the observed slack $(\Delta t, \Delta e)$ when a job completes early:

Option 1: Save Both

Release the next job immediately. Carry Δt and Δe forward for future jobs. *Conservative — bank the slack.*

Option 2: Spend Both on Optional Segments

Use Δt (for execution time) and Δe (for energy) to run additional optional segments on the upcoming job. *Immediate utility gain.*

Option 3: Time $\rightarrow$ Energy Conversion

Use Δt to run a job at a *lower* frequency. This takes more time (consumes Δt) but, in a standard DVFS model, saves energy (increases Δe for later). *Convert perishable time into bankable energy.*

Option 4: Energy $\rightarrow$ Time Conversion

Use Δe to run a job at a *higher* frequency. This costs energy (consumes Δe) but finishes faster, creating time slack Δt for the near future. *Convert bankable energy into perishable time.*

These four options are not isolated choices — they are different ways of *splitting* the available $(\Delta t, \Delta e)$ between the current job and all future jobs. Deciding the split optimally is exactly what the DP does.

2. The Joint DP Formulation

2.1 Why a Single Joint Table

A natural first instinct is to handle energy and time separately — one DP for energy, a greedy rule for time. This fails because the two resources are **coupled**: running an optional segment consumes *both* time and energy simultaneously, and converting between them (Options 3 and 4) trades one for the other. A decision optimal in energy alone may be infeasible in time, and vice versa.

The solution is a single table whose state captures both resources at once.

The 3-Dimensional DP State

$\mathcal{V}[j][\Delta t][\Delta e]$ = maximum additional utility achievable from job j onward given Δt time slack and Δe energy slack available to distribute among jobs $j, j+1, \dots, N$.

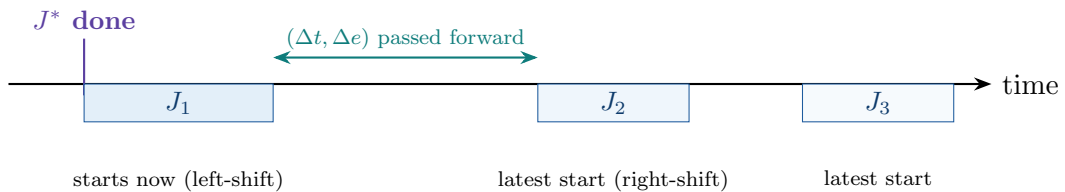
Three dimensions: j = job index in execution order; Δt = time slack; Δe = energy slack.

2.2 The Geometry: Left-Shift J_1 , Right-Shift the Rest

The paper prescribes a specific spatial arrangement when reasoning about the distribution:

“Consider jobs from right to left. When job J_1 is under consideration, assume it is left-shifted while subsequent jobs are right-shifted.”

This means: the job immediately after J^* (J_1) is pulled *left* so it can start using slack *right now*. All later jobs ($J_2, J_3, \dots$) sit at their *latest* feasible start times (right-shifted), so any slack passed to them sits as idle time *before* them — available for a future decision.



The decision for J_1 is therefore: *how much of $(\Delta t, \Delta e)$ to consume on J_1 itself, and how much to pass forward to $J_2, J_3, \dots$?*

2.3 The Recurrence

DP Recurrence (built right to left)

$$\mathcal{V}[j][\Delta t][\Delta e] = \max_{(a_t, a_e)} \left\{ \text{gain}(j, a_t, a_e) + \mathcal{V}[j+1][\Delta t - a_t][\Delta e - a_e] \right\}$$

where (a_t, a_e) is the slack allocated to job j , subject to $0 \leq a_t \leq \Delta t$ and $0 \leq a_e \leq \Delta e$.

The term $\text{gain}(j, a_t, a_e)$ is the utility job j achieves when granted a_t extra time units and a_e extra energy units — it runs as many additional optional segments as both budgets allow.

2.4 Mapping the Four Options to (a_t, a_e)

Each of the paper's four options corresponds to a specific allocation pattern in the recurrence:

Option	Allocation	Effect
1. Save	$a_t = 0, a_e = 0$	Pass full $(\Delta t, \Delta e)$ to J_2 . Gain = 0 now.
2. Spend	$a_t > 0, a_e > 0$	Run optional segs on J_1 . Gain = $u \times \text{seg sizes}$.
3. T→E	$a_t > 0, a_e < 0$	J_1 runs slower; a_e <i>negative</i> returns energy to the pool.
4. E→T	$a_t < 0, a_e > 0$	J_1 runs faster; a_t <i>negative</i> returns time to the future.

Negative allocations ($a_t < 0$ or $a_e < 0$) are how the conversion options enter the recurrence: a negative allocation *returns* that resource to the pool rather than consuming it. Options 3 and 4 are slack-conversion moves, not slack-spending moves.

2.5 Non-Uniform Segment Sizes

A crucial modelling point: optional segments are **not identical**. Each segment k of task i has its own size $e_{i,k}^o$. The cost and gain of adding the next segment depend on which segment it is:

Per-Segment Costs (non-uniform)

$$\begin{aligned} \text{time cost of segment } k &= e_{i,k}^o / f_z = \text{cum}[i][k] - \text{cum}[i][k-1] \text{ (at } f_z) \\ \text{energy cost of segment } k &= E(i, k, z) - E(i, k-1, z) \\ \text{utility gain of segment } k &= u_i \times e_{i,k}^o \end{aligned}$$

For example, a task with optional segments of sizes $[2, 1]$ gains $u_i \times 2$ for the first and $u_i \times 1$ for the second — different absolute gains at different costs, even though the per-unit utility rate u_i is constant. The DP must use the correct $e_{i,k}^o$ at each step; treating all segments as equal would produce wrong table values.

3. Worked Example (3 Jobs, Non-Uniform Segments)

This section traces the full table construction so the mechanics are concrete. Single processor, EDF order $J_1 \rightarrow J_2 \rightarrow J_3$, energy budget for optional work = 8 units (energy simplified to equal execution time).

3.1 Job Parameters

Job	u	Mandatory	Optional segs (sizes)	Notes
J_1	3.0	4	seg1 = 2, seg2 = 1	large then small
J_2	1.0	5	seg1 = 3	one large seg
J_3	2.0	3	seg1 = 1, seg2 = 3	small then large

Actions (with cumulative time and energy cost, and utility gain):

Job	Action	a_t	a_e	gain
J_1	skip / add s1 / add s1+s2	0 / 2 / 3	0 / 2 / 3	0 / 6.0 / 9.0
J_2	skip / add s1	0 / 3	0 / 3	0 / 3.0
J_3	skip / add s1 / add s1+s2	0 / 1 / 4	0 / 1 / 4	0 / 2.0 / 8.0

3.2 Table $\mathcal{V}[3]$ — Last Job

With no future job, $\mathcal{V}[3][\Delta t][\Delta e]$ is just the best feasible action for J_3 alone. The two segments (1 then 3) create two thresholds: a small jump at $(\Delta t=1, \Delta e=1)$ where seg1 fits, and a large jump at $(\Delta t=4, \Delta e=4)$ where both fit.

$\mathcal{V}[3]$	$\Delta e=0$	1	2	3	4	5	6	7	8
$\Delta t=0$	0	0	0	0	0	0	0	0	0
1	0	2	2	2	2	2	2	2	2
2	0	2	2	2	2	2	2	2	2
3	0	2	2	2	2	2	2	2	2
4	0	2	2	2	8	8	8	8	8
5	0	2	2	2	8	8	8	8	8

3.3 Table $\mathcal{V}[2]$ — Job 2 (uses $\mathcal{V}[3]$)

For each $(\Delta t, \Delta e)$: compare *skip* (giving everything to J_3) against *add J_2 's segment* (cost 3, 3, gain 3.0, then pass the rest to J_3).

$\mathcal{V}[2]$	$\Delta e=0$	1	2	3	4	5	6	7	8
$\Delta t=0$	0	0	0	0	0	0	0	0	0
1	0	2	2	2	2	2	2	2	2
2	0	2	2	2	2	2	2	2	2
3	0	2	2	3	3	3	3	3	3
4	0	2	2	3	8	8	8	8	8
5	0	2	2	3	8	8	8	8	8

At $(\Delta t=4, \Delta e=4)$ the table chooses **skip** J_2 (value 8) over adding J_2 's segment (which would give $3 + \mathcal{V}[3][1][1] = 3 + 2 = 5$). The non-uniform sizes make J_3 's two segments together (worth 8.0) more valuable than J_2 's single segment (worth 3.0). The DP discovers this automatically.

3.4 Table $\mathcal{V}[1]$ — Job 1 (uses $\mathcal{V}[2]$)

Three actions now: skip, add s1 (cost 2, 2, gain 6), add s1+s2 (cost 3, 3, gain 9).

$\mathcal{V}[1]$	$\Delta e=0$	1	2	3	4	5	6	7	8
$\Delta t=0$	0	0	0	0	0	0	0	0	0
1	0	2	2	2	2	2	2	2	2
2	0	2	6	6	6	6	6	6	6
3	0	2	6	9	9	9	9	9	9
4	0	2	6	9	11	11	11	11	11
5	0	2	6	9	11	11	11	11	11

The entry $\mathcal{V}[1][4][4] = 11$ is reached by: add both of J_1 's segments (9.0, cost 3, 3), then pass ($\Delta t=1, \Delta e=1$) to J_2 (skip) and J_3 (add seg1, +2.0). Total = 11.0.

3.5 Online Trace at ($\Delta t=4, \Delta e=5$)

At runtime J^* completes early; observed slack is ($\Delta t=4, \Delta e=5$). The action table $\mathcal{A}$ (built alongside $\mathcal{V}$) drives the decisions:

Event	Lookup	Action	Gain	Remaining
J_1	$\mathcal{A}[1][4][5]$	add s1+s2	9.0	($\Delta t=1, \Delta e=2$)
J_2	$\mathcal{A}[2][1][2]$	skip (needs $\Delta t \geq 3$)	0	($\Delta t=1, \Delta e=2$)
J_3	$\mathcal{A}[3][1][2]$	add s1 (size 1)	2.0	($\Delta t=0, \Delta e=1$)
Total utility			11.0	matches $\mathcal{V}[1][4][5]$

Each online event is a single table lookup plus a feasibility check against the actually-observed time slack — microseconds of work.

4. Second Worked Example (4 Jobs, Verified)

The first example showed the mechanics. This larger example shows how the DP makes *globally* optimal trade-offs across four jobs with very different utility rates and segment structures. **Every table cell and every trace below was computed and verified programmatically.**

Single processor, EDF order $J_1 \rightarrow J_2 \rightarrow J_3 \rightarrow J_4$. Energy simplified to equal execution time (at $f_{\max}$), as before. State ranges: $\Delta t \in [0, 6]$, $\Delta e \in [0, 7]$.

4.1 Job Parameters

Job	u	Optional segs (sizes)	Character
J_1	2.0	seg1 = 1, seg2 = 2	low utility, small-then-large
J_2	4.0	seg1 = 2	highest utility, one seg
J_3	1.0	seg1 = 1, seg2 = 1, seg3 = 1	lowest utility, three equal segs
J_4	3.0	seg1 = 3	high utility, one large seg

Action set (cumulative time/energy cost = segment-size sum; gain = $u \times$ size sum):

Job	Actions ($k = 0, 1, \dots$)	a_t	a_e	gain
J_1	skip / s1 / s1+s2	0 / 1 / 3	0 / 1 / 3	0 / 2.0 / 6.0
J_2	skip / s1	0 / 2	0 / 2	0 / 8.0
J_3	skip / s1 / s1+s2 / s1+s2+s3	0 / 1 / 2 / 3	0 / 1 / 2 / 3	0 / 1.0 / 2.0 / 3.0
J_4	skip / s1	0 / 3	0 / 3	0 / 9.0

4.2 Table $\mathcal{V}[4]$ — Last Job (J_4)

J_4 has a single segment of size 3, so its only non-trivial action needs $\Delta t \geq 3$ and $\Delta e \geq 3$, yielding gain 9.0.

$\mathcal{V}[4]$	$\Delta e=0$	1	2	3	4	5	6	7
$\Delta t=0$	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0
3	0	0	0	9	9	9	9	9
4	0	0	0	9	9	9	9	9
5	0	0	0	9	9	9	9	9
6	0	0	0	9	9	9	9	9

4.3 Table $\mathcal{V}[3]$ — Job 3 (uses $\mathcal{V}[4]$)

J_3 has three small equal segments ($u = 1.0$). Each adds gain 1.0. But every unit of slack spent on J_3 is a unit denied to J_4 (gain rate 3.0). The table shows the DP weighing J_3 's cheap-but-low-value segments against saving for J_4 .

$\mathcal{V}[3]$	$\Delta e=0$	1	2	3	4	5	6	7
$\Delta t=0$	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1
2	0	1	2	2	2	2	2	2
3	0	1	2	9	9	9	9	9
4	0	1	2	9	10	10	10	10
5	0	1	2	9	10	11	11	11
6	0	1	2	9	10	11	12	12

At $(\Delta t=3, \Delta e=3)$ the DP value is **9**, not 3. Compare: spending all of $(3, 3)$ on J_3 's three segments gives 3.0; *skipping* J_3 entirely and passing $(3, 3)$ to J_4 gives $0 + \mathcal{V}[4][3][3] = 9.0$. The DP correctly skips the low-utility job to fund the high-utility one. Only when slack exceeds what J_4 needs (e.g. $\Delta t=4, \Delta e=4 \rightarrow 10$) does J_3 get a segment on top.

4.4 Table $\mathcal{V}[2]$ — Job 2 (uses $\mathcal{V}[3]$)

J_2 is the *highest*-utility job ($u = 4.0$), one segment of size 2, gain 8.0.

$\mathcal{V}[2]$	$\Delta e=0$	1	2	3	4	5	6	7
$\Delta t=0$	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1
2	0	1	8	8	8	8	8	8
3	0	1	8	9	9	9	9	9
4	0	1	8	9	10	10	10	10
5	0	1	8	9	10	17	17	17
6	0	1	8	9	10	17	18	18

Look at $(\Delta t=5, \Delta e=5) \rightarrow \mathbf{17}$. This is J_2 's segment (8.0, cost 2, 2) *plus* the remaining $(3, 3)$ funding J_4 's segment via $\mathcal{V}[3][3][3] = 9.0$. So $8 + 9 = 17$. The DP packs both high-utility jobs (J_2 and J_4) and skips the low-utility J_3 entirely — a global decision no greedy rule would reliably find.

4.5 Table $\mathcal{V}[1]$ — Job 1, the Entry Point (uses $\mathcal{V}[2]$)

J_1 has $u = 2.0$, segments of size 1 then 2.

$\mathcal{V}[1]$	$\Delta e=0$	1	2	3	4	5	6	7
$\Delta t=0$	0	0	0	0	0	0	0	0
1	0	2	2	2	2	2	2	2
2	0	2	8	8	8	8	8	8
3	0	2	8	10	10	10	10	10
4	0	2	8	10	11	11	11	11
5	0	2	8	10	11	17	17	17
6	0	2	8	10	11	17	19	19

The top-value entry $\mathcal{V}[1][6][6] = \mathbf{19}$ is the most this 4-job system can ever gain from $(\Delta t=6, \Delta e=6)$ of slack.

4.6 Online Traces (All Verified)

The action table $\mathcal{A}$ (recorded alongside $\mathcal{V}$) is consulted once per job. Here are three contrasting runtime scenarios, each verified to match its table value exactly.

Trace 1 — abundant slack ($\Delta t=6, \Delta e=7$):

Event	Lookup	Action	Gain	Remaining
J_1	$\mathcal{A}[1][6][7]$	add s1 (size 1)	2.0	$(\Delta t=5, \Delta e=6)$
J_2	$\mathcal{A}[2][5][6]$	add s1 (size 2)	8.0	$(\Delta t=3, \Delta e=4)$
J_3	$\mathcal{A}[3][3][4]$	skip	0	$(\Delta t=3, \Delta e=4)$
J_4	$\mathcal{A}[4][3][4]$	add s1 (size 3)	9.0	$(\Delta t=0, \Delta e=1)$
Total			19.0	$= \mathcal{V}[1][6][7] \checkmark$

All three high/medium-utility jobs (J_1, J_2, J_4) get segments; the lowest-utility J_3 is skipped despite its segments being the cheapest. *Utility rate beats cheapness.*

Trace 2 — moderate, tight slack ($\Delta t=5, \Delta e=5$):

Event	Lookup	Action	Gain	Remaining
J_1	$\mathcal{A}[1][5][5]$	skip	0	$(\Delta t=5, \Delta e=5)$
J_2	$\mathcal{A}[2][5][5]$	add s1 (size 2)	8.0	$(\Delta t=3, \Delta e=3)$
J_3	$\mathcal{A}[3][3][3]$	skip	0	$(\Delta t=3, \Delta e=3)$
J_4	$\mathcal{A}[4][3][3]$	add s1 (size 3)	9.0	$(\Delta t=0, \Delta e=0)$
Total			17.0	$= \mathcal{V}[1][5][5] \checkmark$

The instructive case. The DP *skips* J_1 — the very first job — even though J_1 could afford a segment right now. Spending on J_1 would leave insufficient slack to fund both J_2 (rate 4.0) and J_4 (rate 3.0). By skipping the low-rate J_1 , the DP preserves exactly enough to pack the two most valuable jobs. A greedy “use slack as soon as you have it” policy would take J_1 ’s segment and end up with less total utility. This is precisely the lookahead value the precomputed table encodes.

Trace 3 — plenty of energy, scarce time ($\Delta t=3, \Delta e=7$):

Event	Lookup	Action	Gain	Remaining
J_1	$\mathcal{A}[1][3][7]$	add s1 (size 1)	2.0	$(\Delta t=2, \Delta e=6)$
J_2	$\mathcal{A}[2][2][6]$	add s1 (size 2)	8.0	$(\Delta t=0, \Delta e=4)$
J_3	$\mathcal{A}[3][0][4]$	skip (no time)	0	$(\Delta t=0, \Delta e=4)$
J_4	$\mathcal{A}[4][0][4]$	skip (no time)	0	$(\Delta t=0, \Delta e=4)$
Total			10.0	$= \mathcal{V}[1][3][7] \checkmark$

Time is the binding constraint here, not energy. Four units of energy ($\Delta e=4$) sit unused at the end because all the *time* slack (Δt) was exhausted after J_1 and J_2 . No amount of spare energy can buy J_4 ’s segment when there is no time left to execute it. This is exactly why the DP must track *both* dimensions — an energy-only DP would wrongly believe J_4 was still affordable.

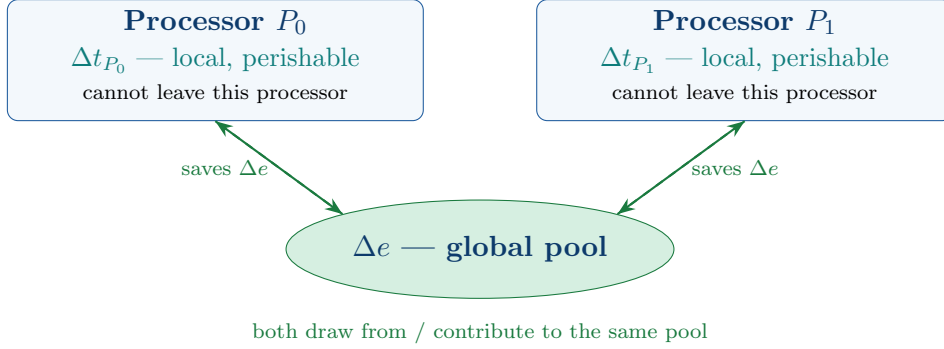
4.7 What This Larger Example Demonstrates

Lesson	Where it shows up
Rate beats cheapness	J_3 's segments are the cheapest (size 1 each) yet are skipped to fund higher-rate J_2, J_4 .
Skip-the-first	Trace 2 skips J_1 entirely — a greedy policy cannot do this.
Both dims matter	Trace 3 ends with unused energy because time ran out — energy-only reasoning would fail.
Global packing	$(\Delta t=5, \Delta e=5)$ packs the two best jobs ($J_2+J_4 = 17$) by skipping the two worst.

5. The Global Energy vs Local Time Asymmetry

This is the deepest structural issue in the online model, so it gets its own section.

5.1 Statement of the Problem



The clean single- Δt DP from the worked example is only valid when all the jobs $\{J_1, J_2, J_3\}$ live on the **same processor**. The moment jobs span multiple processors, each has its own independent Δt_{P_x} , while Δe remains a single shared quantity.

The exact joint state would be:

The Exact (Intractable) State

$$\mathcal{V}[j][\Delta t_{P_0}][\Delta t_{P_1}] \cdots [\Delta t_{P_{m-1}}][\Delta e]$$

State space = $N_{\text{jobs}} \times T^m \times E$ — **exponential in the number of processors m .**

5.2 Three Ways to Tackle the Asymmetry

Approach A — Per-Processor Time DP, Shared Energy Coupling

Keep one DP per processor over its own Δt_{P_x} , but let all of them draw from a single shared energy budget. The energy dimension is the only thing that couples the processors.

Idea: Run m independent time-DPs, one per processor, each indexed $\mathcal{V}_x[j][\Delta t_{P_x}][\Delta e_{\text{share}}]$. The shared Δe is allocated to processors by a higher-level rule (see Approach C). Because time is local, each processor's time reasoning is genuinely independent — only energy must be arbitrated centrally.

Strength: avoids the T^m blow-up — each table is only $N \times T \times E$.

Weakness: the energy split between processors is decided heuristically, not jointly optimally.

Approach B — Distribute the Energy Budget Offline (Per-Processor Energy)

The paper itself raises this option: “Energy constraint can be defined globally or per processor by distributing energy budget.”

Partition the global budget B into per-processor sub-budgets $B_{P_0}, B_{P_1}, \dots$ offline. Then each processor runs a fully independent single- Δt , single- Δe DP — exactly the clean form from the worked example.

	Global energy	Per-processor energy
Coupling	all processors share Δe	each processor independent
DP form	needs joint state	clean $\mathcal{V}[j][\Delta t][\Delta e]$ per proc
Optimality	higher (energy goes where best)	lower (rigid split)
Tractability	hard (coupling)	easy (decoupled)

Strength: makes the problem fully separable — the worked-example DP applies per processor unchanged.

Weakness: a fixed offline split wastes energy when one processor has high-utility work and another has none. The split quality depends on predicting where utility will be — which is exactly what is uncertain online.

Approach C — Utility-Density Arbitration for Shared Energy

Keep energy global, but resolve contention with a precomputed priority. This is the paper’s own suggestion for concurrent completions: “*consider the job which has more higher density in the future — this can be computed offline too.*”

Future Utility Density

$$\rho_{P_x}(t) = \sum_{\substack{\text{jobs } (i,j) \text{ on } P_x \\ \text{with } r_{i,j} \geq t}} \frac{u_i \times (\text{remaining optional work})}{p_i}$$

The processor with higher ρ has more valuable upcoming work, so it gets first claim on shared energy slack.

Strength: keeps energy global (no wasted budget), resolves conflicts cheaply with an offline-precomputed ordering.

Weakness: greedy — it can mis-prioritise when one processor has a single high-density job versus another with several medium-density jobs whose combined value is larger.

5.3 Recommended Combination

Practical recommendation: Use **Approach A** (per-processor time DP) for the time dimension — it is exact and cheap because time is genuinely local. Layer **Approach C** (utility-density arbitration) on top to allocate the global energy pool when multiple processors compete. Reserve **Approach B** (offline energy split) as a fallback for systems where even the arbitration overhead is unacceptable, accepting some lost optimality for full decoupling.

6. Top 5 Concerns & Suggested Solutions

This section collects the most consequential issues in the online model, with concrete suggested fixes.

Issue 1 — Energy Conversion Options Contradict the Default Energy Model

The paper’s Options 3 and 4 assume that lowering frequency saves energy. But with the project’s default energy parameters $\alpha = 1$, $\beta = 0.5$, the energy function $E = \alpha(c/f) + \beta f^2 c$ is *minimised at* $f_{\max}$:

$$f_{\text{opt}} = (\alpha/2\beta)^{1/3} = 1.0 = f_{\max}.$$

Lowering frequency *increases* energy, so Option 3 reduces Δe rather than increasing it, and Option 4 cannot raise frequency beyond $f_{\max}$. Half the action set is inert or harmful.

Fix: Either (a) recognise that under $\alpha=1, \beta=0.5$ the online action set collapses to Options 1 and 2 only (save, or spend on segments) and build the DP with just those — simpler and correct; or (b) if frequency conversion is genuinely wanted, re-parameterise to a leakage-dominated model (α small, β large) where $f_{\text{opt}} < f_{\max}$ and the conversion options become meaningful. Choose (a) unless conversion is a required feature.

Issue 2 — DP Built from $k=0$ Instead of the Offline-Assigned k

The offline schedule already assigns each job a segment count k_{off} . At runtime the online phase should add segments *beyond* k_{off} . If the DP is built assuming the next segment is segment 1 (i.e. from $k=0$), every cost and gain is wrong for jobs that already carry optional segments offline.

Fix: Build the DP action set for each job starting at its offline-assigned k_{off} . The first online action for a job adds segment $k_{\text{off}}+1$ with cost $e_{i, k_{\text{off}}+1}^o$ and gain $u_i \times e_{i, k_{\text{off}}+1}^o$, not segment 1. The baseline energy/time already consumed by the offline segments is subtracted before computing online slack.

Issue 3 — Multi-Processor Time Slack Cannot Share One Δt Dimension

Time slack is processor-local; energy is global. A single Δt in the DP state is only valid when all jobs sit on one processor. The exact multi-processor state $\mathcal{V}[j][\Delta t_{P_0}] \cdots [\Delta t_{P_{m-1}}][\Delta e]$ is exponential in m and intractable.

Fix: Decouple time from energy (Section 5). Run one time-DP per processor over its local Δt_{P_x} (Approach A), and arbitrate the shared global Δe with precomputed utility-density priority (Approach C). This keeps each table at $N \times T \times E$ and avoids the T^m explosion while preserving exact local-time reasoning.

Issue 4 — Δt Is Not a Scalar Under Preemptive EDF

Under preemption the true timing constraint is the DBF window condition, not a single “free time before the next job” scalar. Adding optional work to one job raises the demand in *every* DBF window containing it — affecting non-consecutive jobs that share those windows, not just the immediate successor in the chain.

Fix: Interpret the DP’s Δt as consumption of the *binding DBF window slack* for the job, i.e. Δt tracks how much of min-window-slack has been used. The feasibility test for adding a segment becomes $a_t \leq \text{min_window_slack}(j)$ at the current state — exactly the `min_slack_for_job` computation already used in the heuristic. This makes the scalar Δt a faithful proxy for the window constraint.

Issue 5 — Loose Bounds Make the Table Intractable

The paper’s theoretical bounds $\Delta e \leq B$ and $\Delta t < d_1$ are far too loose. With B in the tens of thousands and d_1 in the hundreds, the table $N \times \Delta t_{\max} \times \Delta e_{\max}$ has billions of entries — neither full DP nor Lazy DP is feasible without tighter bounds.

Fix (the paper’s own “better bounds from schedule”): Bound Δt per job by $\text{min_window_slack}(j)$ at the mandatory baseline — typically only a handful of time units, not d_1 . Bound Δe by $E_{\text{init}}^{\text{slack}} = B - \sum E(\text{mandatory}, f_{\max})$, and per-event by the WCET–ACET energy difference. These schedule-derived ceilings shrink the table from billions to thousands of entries and are exactly what makes **Lazy DP** viable: at runtime, clip computation to $\min(\Delta t_{\text{obs}}, \Delta t_{\text{bound}}) \times \min(\Delta e_{\text{obs}}, \Delta e_{\text{bound}})$.

7. Efficiency: Making the DP Practical

The paper lists three improvements. Each addresses a different cost.

7.1 Better Bounds

Covered in Issue 5. Schedule-derived ceilings on Δt and Δe are the foundation — without them nothing else is tractable. The key quantities, $\text{min_window_slack}(j)$ and $E_{\text{init}}^{\text{slack}}$, are already computed by the offline heuristic, so they come essentially for free.

7.2 Aggregate States

Bucket nearby slack values rather than tracking every unit:

$$\Delta e \in \{0, 1, \dots, 100\} \longrightarrow \Delta e \in \{\text{low}, \text{med}, \text{high}\}.$$

The table shrinks by the bucketing factor; the cost is bounded sub-optimality at bucket boundaries. The right granularity depends on the segment-size distribution — finer where segments are small, coarser where they are large.

7.3 Lazy DP

Do not precompute the entire table offline. Store only the recurrence and the bounds. At runtime, given observed $(\Delta t_{\text{obs}}, \Delta e_{\text{obs}})$, compute *only* the reachable entries:

$$\Delta t \in [0, \min(\Delta t_{\text{obs}}, \Delta t_{\text{bound}})], \quad \Delta e \in [0, \min(\Delta e_{\text{obs}}, \Delta e_{\text{bound}})].$$

	Full Precomputed DP	Lazy DP
Offline cost	build entire table	store bounds + recurrence only
Online cost	one lookup (microseconds)	small targeted computation
Memory	large ($N \times T \times E$)	minimal
Best when	slack range is small & bounded	slack is usually small, occasionally large

The tighter the schedule-derived bounds, the smaller the Lazy DP computation region — which is why Issue 5 (bounds) is the linchpin: it makes both the full DP and Lazy DP practical simultaneously.

8. Summary

Concept	Takeaway
State	$\mathcal{V}[j][\Delta t][\Delta e]$ — job index, time slack, energy slack.
Construction	Right to left; reuse $\mathcal{V}[j+1]$ for $\mathcal{V}[j]$; record action table $\mathcal{A}$.
Online use	One lookup per job completion, filtered by observed time feasibility.
Four options	Save / spend / time→energy / energy→time — but conversions are inert under $\alpha=1, \beta=0.5$.
Segments	Non-uniform sizes; per-segment cost and gain must use $e_{i,k}^o$.
Asymmetry	Energy global (DP dimension), time local (per-processor). Decouple time-DPs, arbitrate energy by utility density.
Tractability	Schedule-derived bounds + Lazy DP + aggregate states.

The one-line essence: Precompute, offline and right-to-left, a table that says “given this much time and energy slack, here is the best total utility from here on and the action that achieves it” — then at runtime simply observe the actual slack, look up the action, check it fits the locally-available time, and execute. The hard reasoning happens once, offline; the online phase is reduced to table lookups.